



Tribhuvan University

Faculty of Humanities and Social Sciences

Kirtipur, Kathmandu

A Lab Report

On

Programming Logic & Techniques using Python (MCA504)

Submitted by

Name: Bishal Rijal

Roll No: 6 (Six)

Tu Register no: 6-2-1133-50-2019

Submitted to

Roshan Tandukar

Patan Multiple Campus

Patandhoka, Lalitpur

Department of MCA

Affiliated to Tribhuvan University

Sign of Internal

.....

Sign of External

.....

September- 2026

Contents

Lab 3: 1. Perform slicing operations on a 2-dimensional list of size 3*3.....	1
2. Create a list with the ordered pair {(2,3),(1,4),(4,2),(5,6),(6,2)} and a function that returns the sum of two numbers. Use list comprehension to get the sum of each ordered pairs.(Samples are on the slides)	2
3. Create a file with name Sample.txt and write some contents into it. Also read the contents of the file. Also copy the contents of the file to another file with name Copy.txt.	3
Lab 4: 1. Create a class named Student which should contain the faculty, a list of subjects in a dictionary and an initializer. Create two objects of Student class and display the values of objects.	4
2. Write an example for method overloading.	5
3. Write a program to overload operators '+' and '-'.	6
Lab 5: 1. Create a GUI form that takes two numbers and displays the greater number in a label.	7
2. Create a GUI form to implement a simple calculator.	9
3. Create a table Product(PId, itemname, price) and insert data to it. Also find the list of products that have price more than Rs. 2000.	10
Lab 6: 1. Create a list in NumPy and perform different types of sorting.	11
2. Create a NumPy array of size 10*10 with all values 1. Assign the non-border values 0 and display it.	12
3. Create two dataframes of your choice using Pandas and perform all the joining operations.	13
4. Use Matplotlib for different data visualization techniques (point plot, line plot, bar graph, histogram, boxplot and 3D plot)	14

Lab 3: 1. Perform slicing operations on a 2-dimensional list of size 3*3.

2-dimensional list of size 3x3

```
matrix = [  
    [1, 2, 3],  
    [4, 5, 6],  
    [7, 8, 9]  
]
```

```
print("Original List:")
```

```
print(matrix)
```

```
print("\nFirst row:")
```

```
print(matrix[0][:])
```

```
print("\nFirst two rows:")
```

```
print(matrix[:2])
```

```
print("\nLast two rows:")
```

```
print(matrix[1:])
```

```
print("\nFirst two columns:")
```

```
print([row[:2] for row in matrix])
```

```
print("\nLast two columns:")
```

```
print([row[1:] for row in matrix])
```

```
print("\nMiddle element:")
```

```
print(matrix[1][1])
```

```
C:\Users\bisha\OneDrive\Desktop\MCA\
Original List:
[[1, 2, 3], [4, 5, 6], [7, 8, 9]]

First row:
[1, 2, 3]

First two rows:
[[1, 2, 3], [4, 5, 6]]

Last two rows:
[[4, 5, 6], [7, 8, 9]]

First two columns:
[[1, 2], [4, 5], [7, 8]]

Last two columns:
[[2, 3], [5, 6], [8, 9]]

Middle element:
5
```

2. Create a list with the ordered pair {(2,3),(1,4),(4,2),(5,6),(6,2)} and a function that returns the sum of two numbers. Use list comprehension to get the sum of each ordered pairs.(Samples are on the slides)

```
pairs = [(2, 3), (1, 4), (4, 2), (5, 6), (6, 2)]
```

```
def add(a, b):  
    return a + b
```

```
sums = [add(a, b) for a, b in pairs]
```

```
print("Ordered Pairs:")  
print(pairs)
```

```
print("Sum of each ordered pair:")  
print(sums)
```

```
C:\Users\bisha\OneDrive\Desktop\MCA\First S  
Ordered Pairs:  
[(2, 3), (1, 4), (4, 2), (5, 6), (6, 2)]  
Sum of each ordered pair:  
[5, 5, 6, 11, 8]
```

3. Create a file with name Sample.txt and write some contents into it. Also read the contents of the file. Also copy the contents of the file to another file with name Copy.txt.

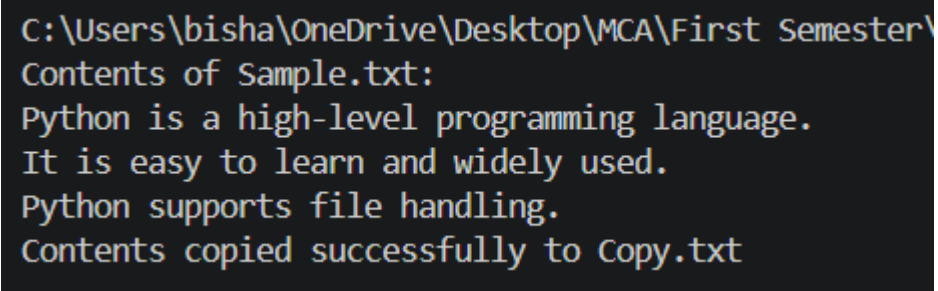
```
# Create and write to Sample.txt
with open("Sample.txt", "w") as file:
    file.write("Python is a high-level programming language.\n")
    file.write("It is easy to learn and widely used.\n")
    file.write("Python supports file handling.")
```

```
# Read Sample.txt
with open("Sample.txt", "r") as file:
    contents = file.read()
```

```
print("Contents of Sample.txt:")
print(contents)
```

```
# Copy contents to Copy.txt
with open("Copy.txt", "w") as file:
    file.write(contents)
```

```
print("Contents copied successfully to Copy.txt")
```



```
C:\Users\bisha\OneDrive\Desktop\MCA\First Semester\
Contents of Sample.txt:
Python is a high-level programming language.
It is easy to learn and widely used.
Python supports file handling.
Contents copied successfully to Copy.txt
```

Lab 4: 1. Create a class named Student which should contain the faculty, a list of subjects in a dictionary and an initializer. Create two objects of Student class and display the values of objects.

```
class Student:
    def __init__(self, faculty, subjects):
        self.faculty = faculty
        self.subjects = subjects
```

```
student1 = Student(
    "MCA",
    {
        "Python": 80,
        "Database": 75,
        "Operating System": 82
    }
)
```

```
student2 = Student(
    "MCA",
    {
        "Python": 85,
        "Database": 88,
        "Operating System": 79
    }
)
```

```
print("Student 1")
print("Faculty:", student1.faculty)
print("Subjects:", student1.subjects)
```

```
print("\nStudent 2")
print("Faculty:", student2.faculty)
print("Subjects:", student2.subjects)
```

```
C:\Users\bisha\OneDrive\Desktop\MCA\First Semester\Python\Lab Task
Student 1
Faculty: MCA
Subjects: {'Python': 80, 'Database': 75, 'Operating System': 82}

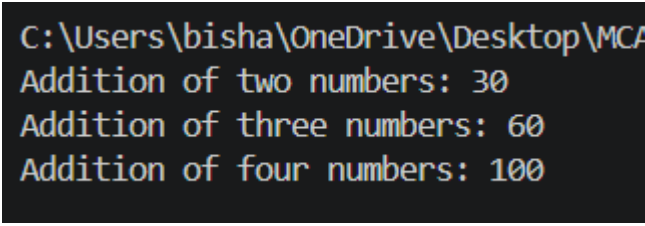
Student 2
Faculty: MCA
Subjects: {'Python': 85, 'Database': 88, 'Operating System': 79}
```

2. Write an example for method overloading.

```
class Calculator:
    def add(self, *numbers):
        return sum(numbers)

c = Calculator()

print("Addition of two numbers:", c.add(10, 20))
print("Addition of three numbers:", c.add(10, 20, 30))
print("Addition of four numbers:", c.add(10, 20, 30, 40))
```



```
C:\Users\bisha\OneDrive\Desktop\MCA
Addition of two numbers: 30
Addition of three numbers: 60
Addition of four numbers: 100
```

3. Write a program to overload operators '+' and '-'.

```
class Number:
```

```
    def __init__(self, value):
```

```
        self.value = value
```

```
    def __add__(self, other):
```

```
        return Number(self.value + other.value)
```

```
    def __sub__(self, other):
```

```
        return Number(self.value - other.value)
```

```
    def __str__(self):
```

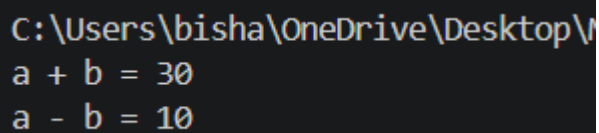
```
        return str(self.value)
```

```
a = Number(20)
```

```
b = Number(10)
```

```
print("a + b =", a + b)
```

```
print("a - b =", a - b)
```



```
C:\Users\bisha\OneDrive\Desktop\
a + b = 30
a - b = 10
```

Lab 5: 1. Create a GUI form that takes two numbers and displays the greater number in a label.

```
import tkinter as tk

def find_greater():
    try:
        num1 = float(entry1.get())
        num2 = float(entry2.get())

        if num1 > num2:
            result.config(text=f"Greater Number: {num1}")
        elif num2 > num1:
            result.config(text=f"Greater Number: {num2}")
        else:
            result.config(text="Both numbers are equal")
    except ValueError:
        result.config(text="Please enter valid numbers")

root = tk.Tk()
root.title("Greater Number")
root.geometry("350x250")

tk.Label(root, text="Enter First Number").pack(pady=5)
entry1 = tk.Entry(root)
entry1.pack()

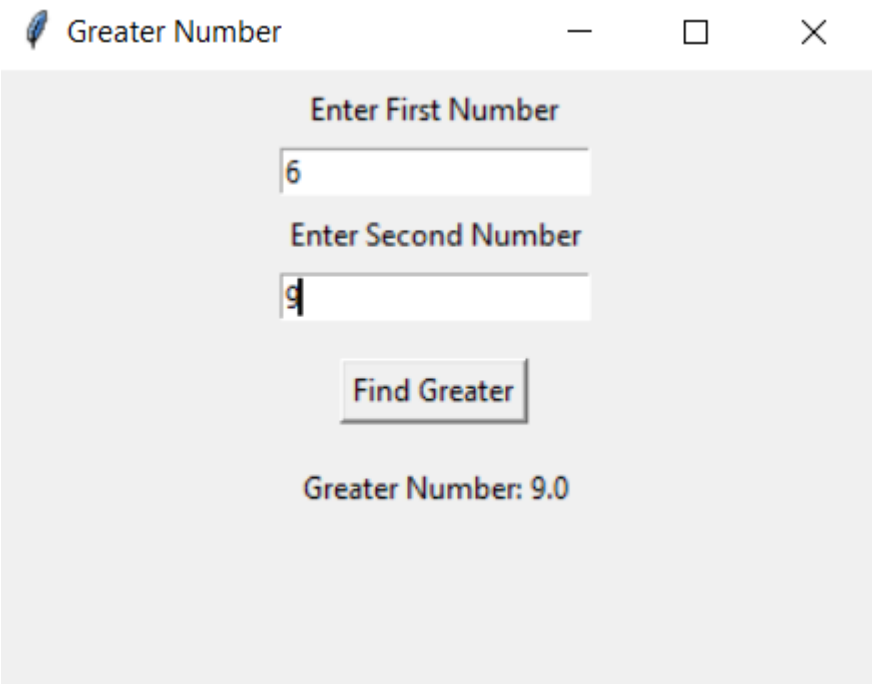
tk.Label(root, text="Enter Second Number").pack(pady=5)
entry2 = tk.Entry(root)
entry2.pack()

tk.Button(root, text="Find Greater", command=find_greater).pack(pady=15)

result = tk.Label(root, text="")
```

result.pack()

root.mainloop()



Greater Number

Enter First Number

6

Enter Second Number

9

Find Greater

Greater Number: 9.0

2. Create a GUI form to implement a simple calculator.

```
import tkinter as tk

def calculate(operator):
    try:
        num1 = float(entry1.get())
        num2 = float(entry2.get())

        if operator == "+":
            result = num1 + num2
        elif operator == "-":
            result = num1 - num2
        elif operator == "*":
            result = num1 * num2
        elif operator == "/":
            if num2 == 0:
                output.config(text="Cannot divide by zero")
                return
            result = num1 / num2

        output.config(text=f"Result: {result}")

    except ValueError:
        output.config(text="Please enter valid numbers")

root = tk.Tk()
root.title("Simple Calculator")
root.geometry("350x300")

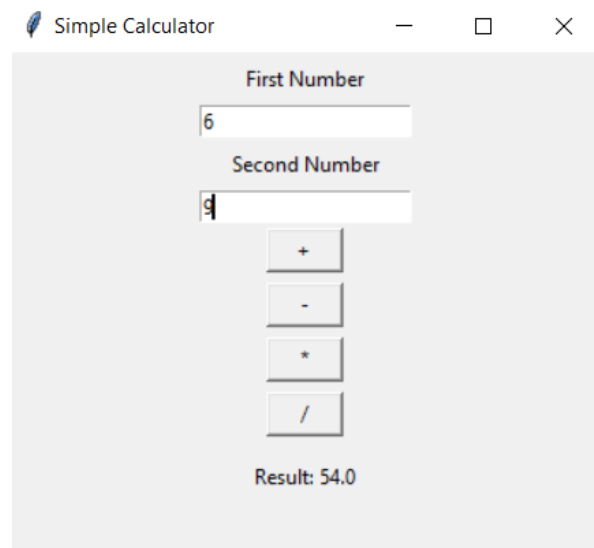
tk.Label(root, text="First Number").pack(pady=5)
entry1 = tk.Entry(root)
entry1.pack()

tk.Label(root, text="Second Number").pack(pady=5)
entry2 = tk.Entry(root)
entry2.pack()

tk.Button(root, text="+", width=5, command=lambda: calculate("+")).pack(pady=3)
tk.Button(root, text="-", width=5, command=lambda: calculate("-")).pack(pady=3)
tk.Button(root, text="*", width=5, command=lambda: calculate("*")).pack(pady=3)
tk.Button(root, text="/", width=5, command=lambda: calculate("/")).pack(pady=3)

output = tk.Label(root, text="")
output.pack(pady=10)

root.mainloop()
```



3. Create a table Product(PId, itemname, price) and insert data to it.

Also find the list of products that have price more than Rs. 2000.

```
import sqlite3

connection = sqlite3.connect("Product.db")
cursor = connection.cursor()

cursor.execute("""
CREATE TABLE IF NOT EXISTS Product(
    PId INTEGER PRIMARY KEY,
    itemname TEXT,
    price REAL
)
""")

cursor.execute("DELETE FROM Product")

products = [
    (1, "Laptop", 75000),
    (2, "Keyboard", 1500),
    (3, "Monitor", 12000),
    (4, "Mouse", 800),
    (5, "Printer", 18000)
]

cursor.executemany(
    "INSERT INTO Product(PId, itemname, price) VALUES (?, ?, ?)",
    products
)

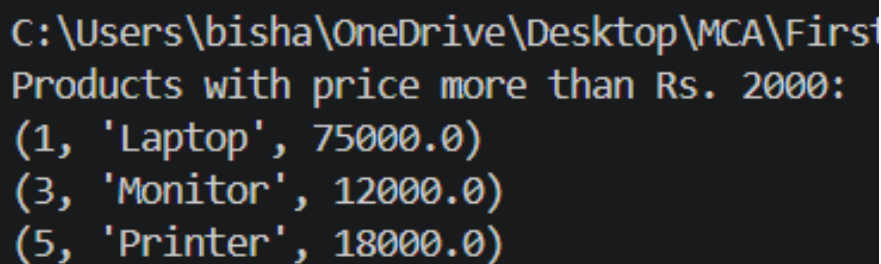
connection.commit()

print("Products with price more than Rs. 2000:")

cursor.execute("SELECT * FROM Product WHERE price > 2000")

for product in cursor.fetchall():
    print(product)

connection.close()
```



```
C:\Users\bisha\OneDrive\Desktop\MCA\First
Products with price more than Rs. 2000:
(1, 'Laptop', 75000.0)
(3, 'Monitor', 12000.0)
(5, 'Printer', 18000.0)
```

Lab 6: 1. Create a list in NumPy and perform different types of sorting.

```
import numpy as np

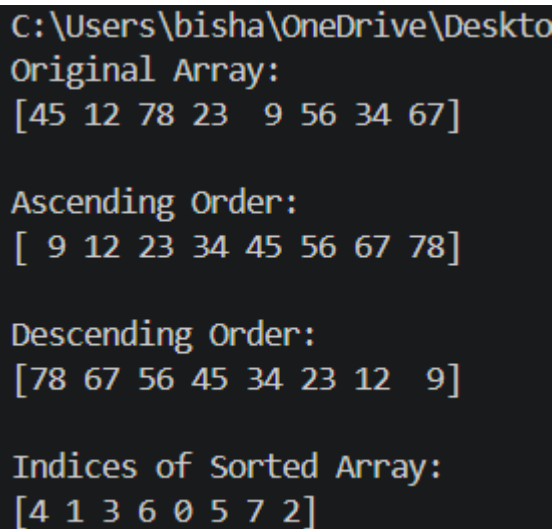
arr = np.array([45, 12, 78, 23, 9, 56, 34, 67])

print("Original Array:")
print(arr)

print("\nAscending Order:")
print(np.sort(arr))

print("\nDescending Order:")
print(np.sort(arr)[::-1])

print("\nIndices of Sorted Array:")
print(np.argsort(arr))
```



C:\Users\bisha\OneDrive\Desktop
Original Array:
[45 12 78 23 9 56 34 67]

Ascending Order:
[9 12 23 34 45 56 67 78]

Descending Order:
[78 67 56 45 34 23 12 9]

Indices of Sorted Array:
[4 1 3 6 0 5 7 2]

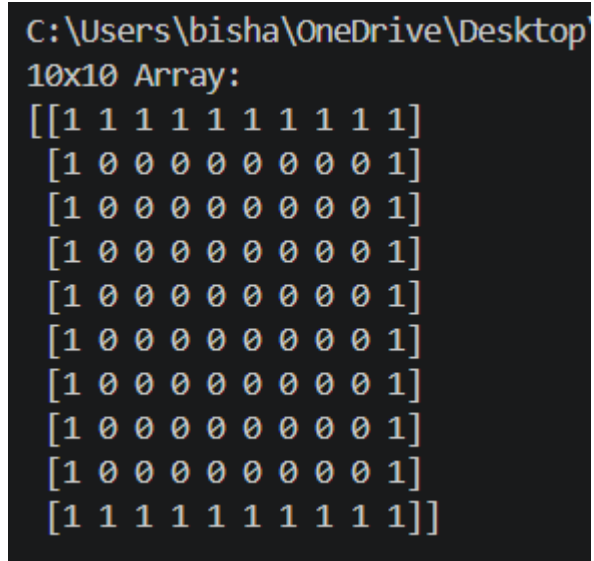
2. Create a NumPy array of size 10*10 with all values 1. Assign the non-border values 0 and display it.

```
import numpy as np

arr = np.ones((10, 10), dtype=int)

arr[1:-1, 1:-1] = 0

print("10x10 Array:")
print(arr)
```



```
C:\Users\bisha\OneDrive\Desktop
10x10 Array:
[[1 1 1 1 1 1 1 1 1 1]
 [1 0 0 0 0 0 0 0 0 1]
 [1 0 0 0 0 0 0 0 0 1]
 [1 0 0 0 0 0 0 0 0 1]
 [1 0 0 0 0 0 0 0 0 1]
 [1 0 0 0 0 0 0 0 0 1]
 [1 0 0 0 0 0 0 0 0 1]
 [1 0 0 0 0 0 0 0 0 1]
 [1 0 0 0 0 0 0 0 0 1]
 [1 1 1 1 1 1 1 1 1 1]]
```

3. Create two dataframes of your choice using Pandas and perform all the joining operations.

```
import pandas as pd

df1 = pd.DataFrame({
    "Id": [1, 2, 3, 4],
    "Name": ["Ram", "Shyam", "Hari", "Sita"]
})

df2 = pd.DataFrame({
    "Id": [3, 4, 5, 6],
    "Marks": [75, 85, 90, 80]
})

print("DataFrame 1:")
print(df1)

print("\nDataFrame 2:")
print(df2)

print("\nInner Join:")
print(pd.merge(df1, df2, on="Id", how="inner"))

print("\nLeft Join:")
print(pd.merge(df1, df2, on="Id", how="left"))

print("\nRight Join:")
print(pd.merge(df1, df2, on="Id", how="right"))

print("\nOuter Join:")
print(pd.merge(df1, df2, on="Id", how="outer"))
```

C:\Users\bisha\OneDrive\Desk

DataFrame 1:

	Id	Name
0	1	Ram
1	2	Shyam
2	3	Hari
3	4	Sita

DataFrame 2:

	Id	Marks
0	3	75
1	4	85
2	5	90
3	6	80

Inner Join:

	Id	Name	Marks
0	3	Hari	75
1	4	Sita	85

Left Join:

	Id	Name	Marks
0	1	Ram	NaN
1	2	Shyam	NaN
2	3	Hari	75.0
3	4	Sita	85.0

Right Join:

	Id	Name	Marks
0	3	Hari	75
1	4	Sita	85
2	5	NaN	90
3	6	NaN	80

Outer Join:

	Id	Name	Marks
0	1	Ram	NaN

4. Use Matplotlib for different data visualization techniques (point plot, line plot, bar graph, histogram, boxplot and 3D plot)

```
import matplotlib.pyplot as plt
import numpy as np

# Point Plot
x = [1, 2, 3, 4, 5]
y = [2, 5, 3, 7, 6]

plt.scatter(x, y)
plt.title("Point Plot")
plt.xlabel("X")
plt.ylabel("Y")
plt.show()

# Line Plot
plt.plot(x, y, marker="o")
plt.title("Line Plot")
plt.xlabel("X")
plt.ylabel("Y")
plt.show()

# Bar Graph
subjects = ["Python", "DBMS", "OS", "Math"]
marks = [80, 75, 85, 70]

plt.bar(subjects, marks)
plt.title("Bar Graph")
plt.xlabel("Subjects")
plt.ylabel("Marks")
plt.show()

# Histogram
data = [12, 15, 18, 20, 22, 22, 25, 27, 28, 30,
        31, 32, 35, 35, 38, 40, 42, 45, 45, 48]

plt.hist(data, bins=5)
plt.title("Histogram")
plt.xlabel("Values")
plt.ylabel("Frequency")
plt.show()

# Boxplot
marks = [55, 60, 65, 70, 72, 75, 78, 80, 85, 90]

plt.boxplot(marks)
plt.title("Boxplot")
plt.ylabel("Marks")
plt.show()

# 3D Plot
```

```
from mpl_toolkits.mplot3d import Axes3D
```

```
x = np.array([1, 2, 3, 4, 5])  
y = np.array([2, 4, 6, 8, 10])  
z = np.array([5, 10, 15, 20, 25])
```

```
fig = plt.figure()  
ax = fig.add_subplot(111, projection="3d")
```

```
ax.scatter(x, y, z)  
ax.set_title("3D Plot")  
ax.set_xlabel("X")  
ax.set_ylabel("Y")  
ax.set_zlabel("Z")
```

```
plt.show()
```

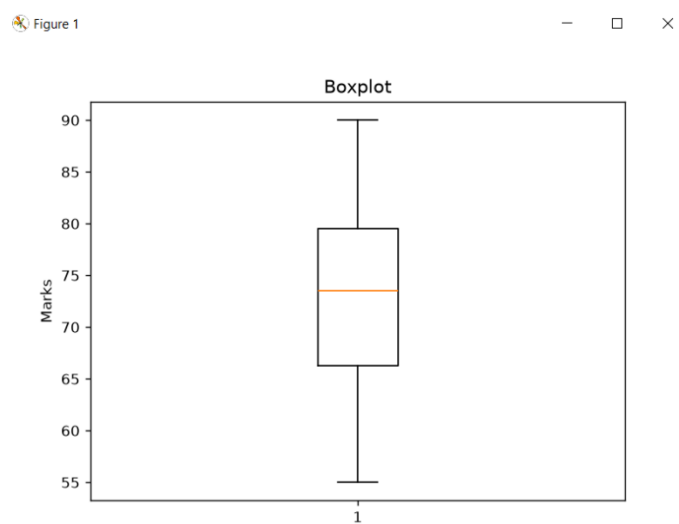
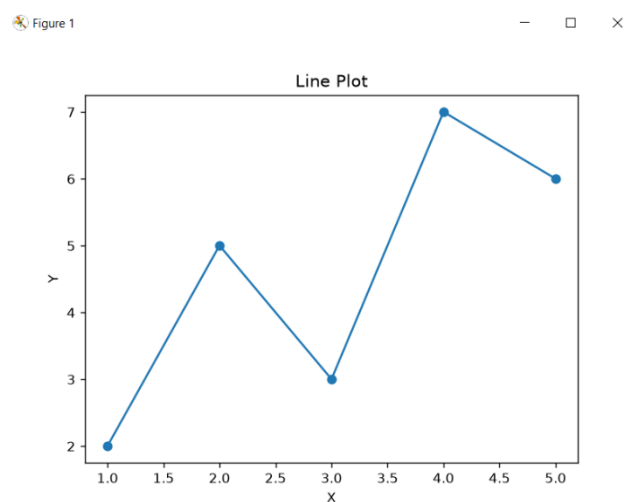
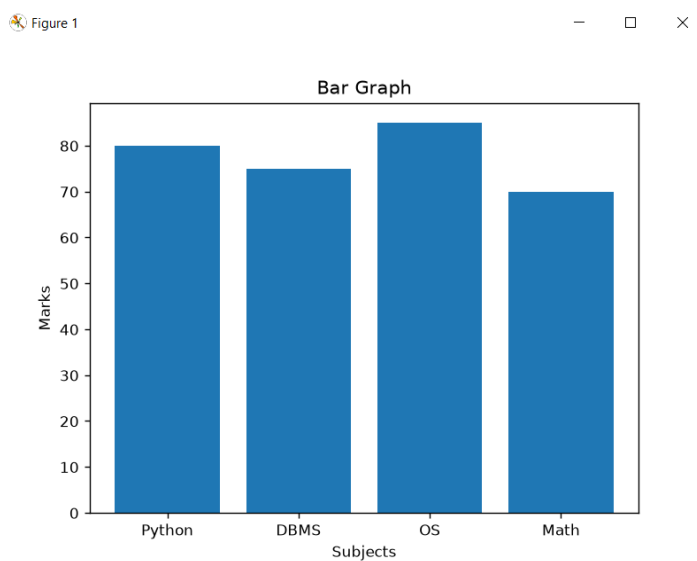
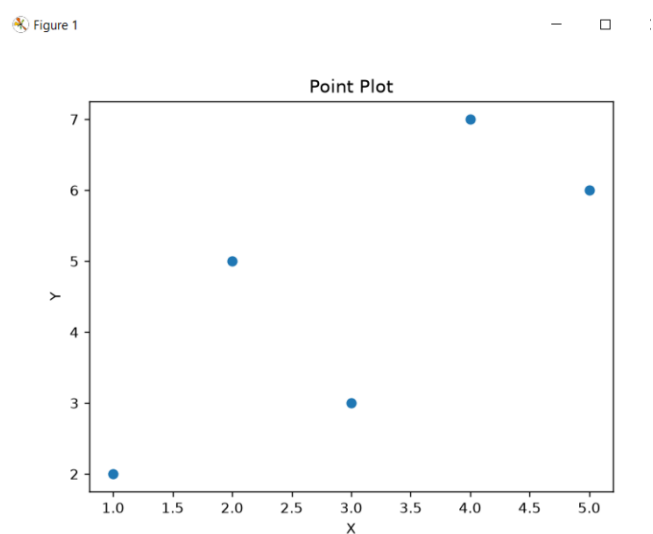


Figure 1

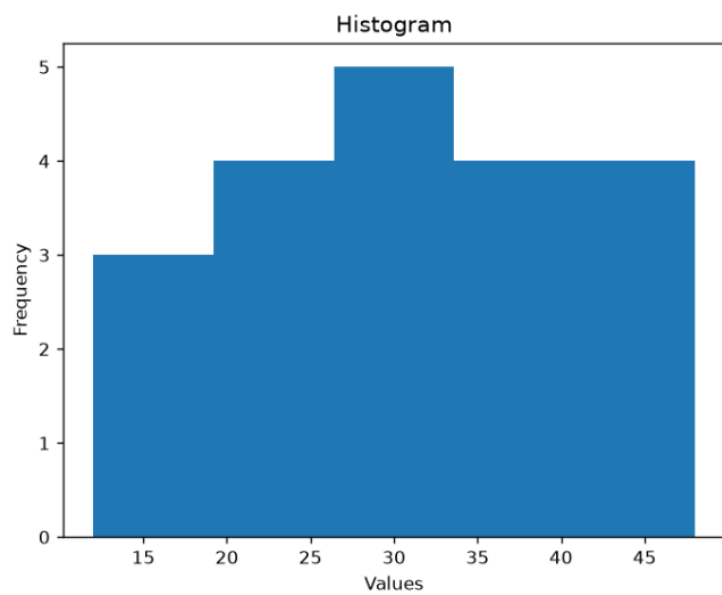


Figure 1

